Sentiment Analysis on Reddit Data

Charles Alders

This project utilizes the Reddit API through the praw Python library to extract the top 570 posts from the r/LifeProTips subreddit. The sentiment scores are generated by TextBlob, and include both the title and body of the Reddit post. After going through some exploratory analysis, I compare the r/LifeProTips subreddit to another less positive subreddit to compare the mean sentiment scores.

Why is this useful?

Comparing sentiments can be a VERY powerful tool for analyzing how masses feel about a product, a company, or an idea. For example, pulling tweets that include a product name and grabbing the most commented on features, and determining whether the customers (or potential customers) are happy with their purchase, or if they feel negatively. Comparing these sentiments to another product or company can give an idea for the competition and the important topics to focus on.

Reddit API to DataFrame

In [1]:
```
import pandas as pd
import praw
import credentials
import nltk
from nltk.corpus import stopwords
```

In [2]:
```
reddit = praw.Reddit(client_id=credentials.client_id,
                     client_secret=credentials.client_secret,
                     user_agent=credentials.user_agent)
```

In [3]:
```
subreddit = reddit.subreddit('LifeProTips').hot(limit=None)
```

In [4]:
```
reddit_data = {
    "title": [],
    "author": [],
    "n_comments": [],
    "score": [],
    "text": [],
    "link": []
}

for post in list(subreddit):
    reddit_data["title"].append(post.title)
    if post.author is None:
        reddit_data["author"].append('')
    else:
        reddit_data["author"].append(post.author.name)

    reddit_data["n_comments"].append(post.num_comments)
    reddit_data["score"].append(post.score)
    reddit_data["text"].append(post.selftext)
    reddit_data["link"].append(post.permalink)
```

In [5]:
```
df = pd.DataFrame(reddit_data)
```

In [6]:
```
df.shape[0]
```

Out[6]:
```
570
```

There are 570 posts contained in the Reddit dataframe.

In [7]:
```
df.head()
```

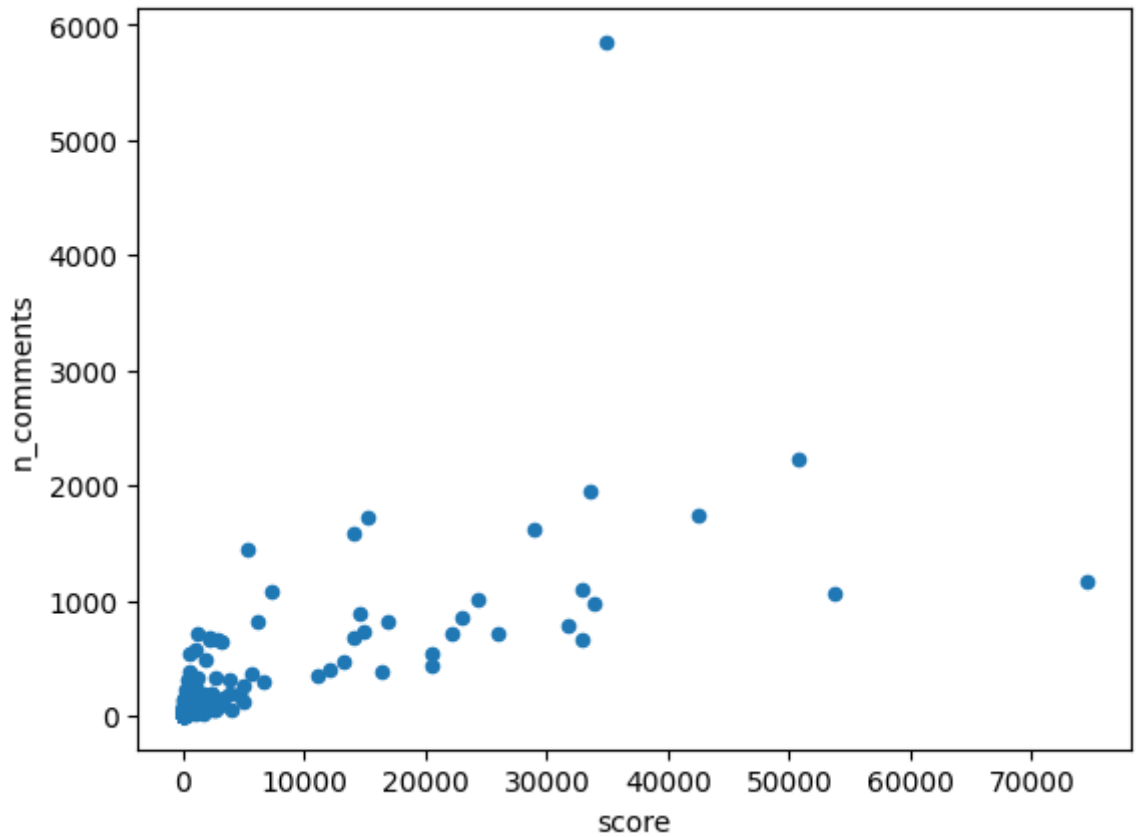
Out[7]:

	title	author	n_comments	score	text	
0	21 million subscribers looking for the real ti...	Minifig81	53	874	From the bottom of the mod team's hearts, here...	/r/LifeProTips/comments/wlgp23/21_milli
1	LPT: Go ahead and take that raise into a highe...	nevertoolate2	1611	28967	Only the money above the old tax bracket will ...	/r/LifeProTips/comments/11iap3h/lpt_go_
2	LPT: If you are used to binge eating or consum...	slapstick15	105	2417	9/10 times this works for me because my body r...	/r/LifeProTips/comments/11i8714/lpt_if_
3	LPT: Ask your landlord to fix stuff a month or...	etvorolim	17	297	If you're planning on leaving a house you rent...	/r/LifeProTips/comments/11ij8wc/lpt_as
4	LPT Never bring stray animals directly home or...	user6876	106	1443	Apparently this is not commonly known by every...	/r/LifeProTips/comments/11i07lu/lpt_ne

Exploratory Data Analysis

In [8]:
```
df.plot.scatter(x="score", y="n_comments")
```

Out[8]:
```
<AxesSubplot:xlabel='score', ylabel='n_comments'>
```



In [9]:
```
df["score"].corr(df["n_comments"])
```

Out[9]:
```
0.7156142491108501
```

As we can see, there is a strong (0.75) positive linear correlation between score and number of comments. I notice one outlier that has what looks like ~5900 comments and only ~35000 score. This will be an interesting post to check out, as I have a feeling it may be controversial.

In [10]:
```
df[(df["score"]>30000) & (df["score"]<40000)]
```

Out[10]:

	title	author	n_comments	score	text	
6	LPT request: is 30 young enough to turn life a...	RationalChaos77	5847	34812	My 37 year old sister says it's too late in li...	/r/LifeProTips/comments/11hivxc
230	LPT: Marry someone who will always have your b...	leelougirl89	969	33958	A stranger gave that advice to my husband whil...	/r/LifeProTips/comments/11bxjgw/l
289	LPT: If someone asks you "how is your son/daug...	Any-Influence5873	660	32934		/r/LifeProTips/comments/119z7mz/
414	LPT: Keep mice out of your house. Two words. S...	mr444guy	1945	33617	I've been fighting mice for 20 years in our ho...	/r/LifeProTips/comments/11789ca/l
455	LPT: for people with low self-esteem. Be as ki...	dr-korbo	791	31738	Edit: Given some of the comments, it seems tha...	/r/LifeProTips/comments/116fmxu
528	LPT: You don't need to find a job you love. Yo...	King-Piece	1100	32981	I don't know if this has been said before by a...	/r/LifeProTips/comments/114migr/

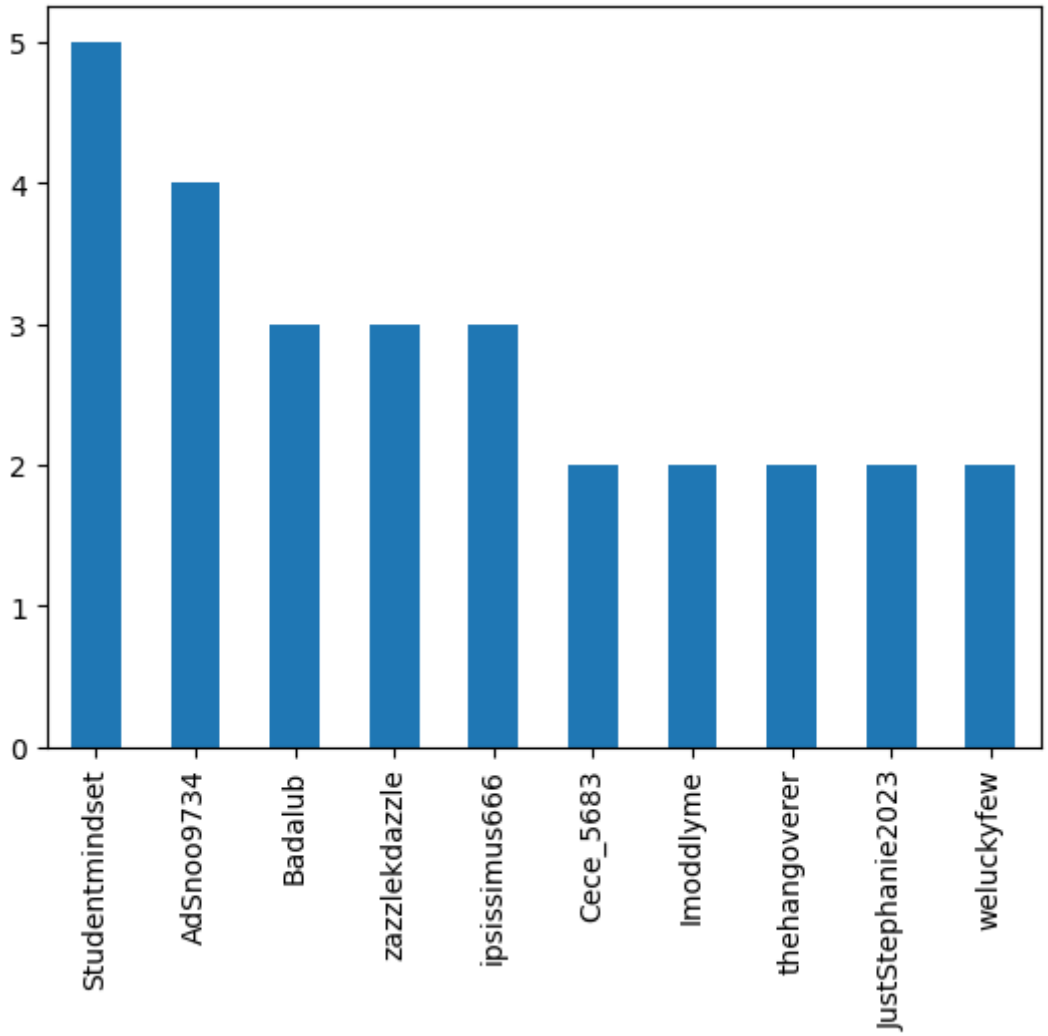
In [11]:
```
print(f'reddit.com{str(df.iloc[6]["link"])}')
```

```
reddit.com/r/LifeProTips/comments/11hivxd/lpt_request_is_30_young_enoug
h_to_turn_life/
```

This post got way more traction than others with a similar score! After looking at the post and the comments, it actually seems to be pretty positive and the increased number of comments are very supportive! This post is about someone turning their life around at age 30. From my quick view of the comments, most of them are sharing similar stories. Basically, the post ended up as more of a discussion than other posts.

In [12]:
```
df["author"].value_counts()[:10].plot.bar()
```

Out[12]: <AxesSubplot:>



In [13]:
```
df.groupby("author")["score"].sum().sort_values(ascending=False)[:10]
```

Out[13]:
```
author
AdSnoo9734           88699
YellowB              53714
ElaborateRuse420     50736
Trick_Gas3677        42547
RationalChaos77      34812
leelougirl89         33958
mr444guy             33617
King-Piece           32981
Any-Influence5873    32934
dr-korbo             31738
Name: score, dtype: int64
```

The spread of posts between authors seems relatively even compared to other subreddits, such as r/science, which has members who have posted 50+ times.

Common Words Distribution using nltk

In [14]:
```
stops = stopwords.words('english')

stops = stops + ["lpt:", "lpt", "request:", "request", "-", "|"]
```

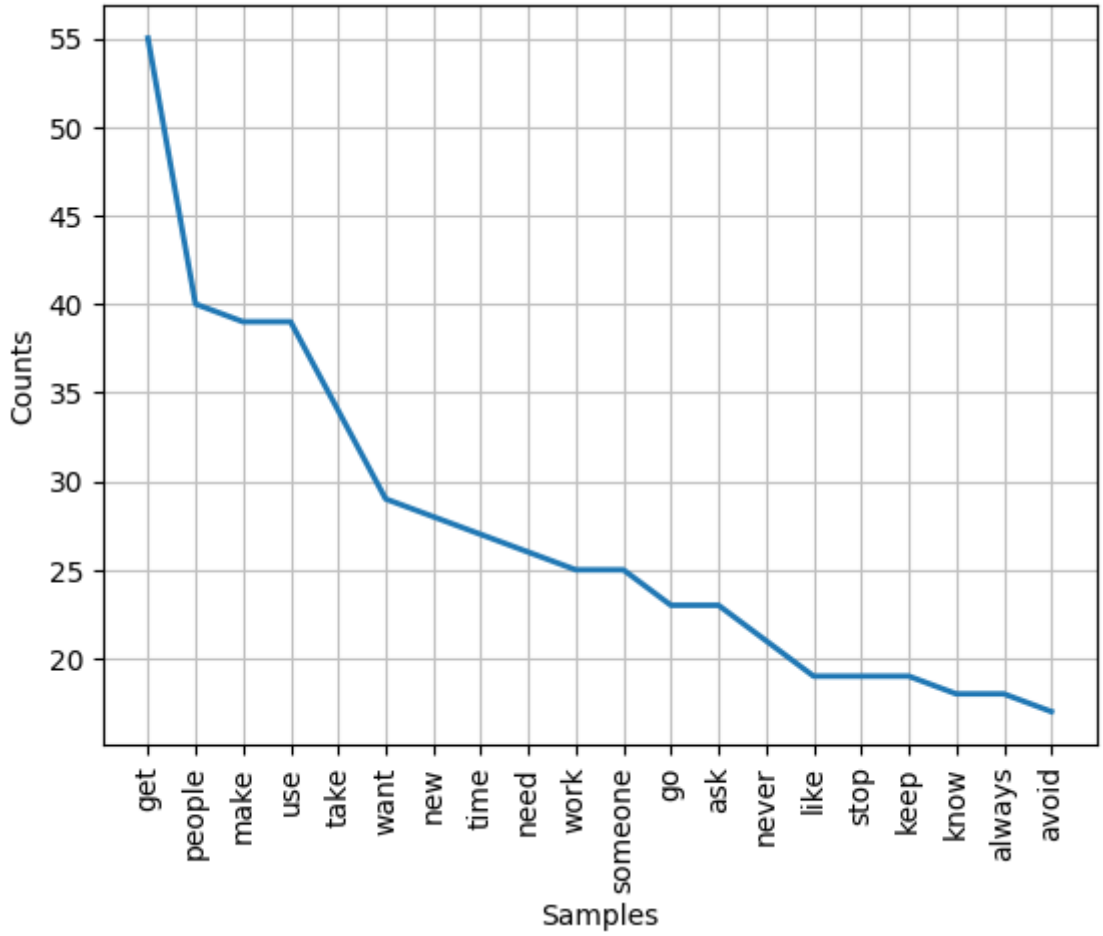
Using stopwords to remove unnecessary words from the freqency distribution. I added a few extras, such as "lpt" and "request", because those are typical words in the LPT subreddit and they do not add any value to this analysis.

In [15]:
```
words = ' '.join(df["title"]).lower().split()
cleaned_words = [w for w in words if w not in set(stops)]
```

In [16]:
```
fd = nltk.FreqDist(cleaned_words)
fd.most_common(20)
```

Out[16]:
```
[('get', 55),
 ('people', 40),
 ('make', 39),
 ('use', 39),
 ('take', 34),
 ('want', 29),
 ('new', 28),
 ('time', 27),
 ('need', 26),
 ('work', 25),
 ('someone', 25),
 ('go', 23),
 ('ask', 23),
 ('never', 21),
 ('like', 19),
 ('stop', 19),
 ('keep', 19),
 ('know', 18),
 ('always', 18),
 ('avoid', 17)]
```

In [17]:
```
fd.plot(20)
```



Out[17]:
```
<AxesSubplot:xlabel='Samples', ylabel='Counts'>
```

Lots of the common words are verbs, which makes sense considering this subreddit is essentially telling people things they can do.

Extracting Sentiment Scores

In [18]:
```
df["title_and_text"] = df["title"] + " " + df["text"]
```

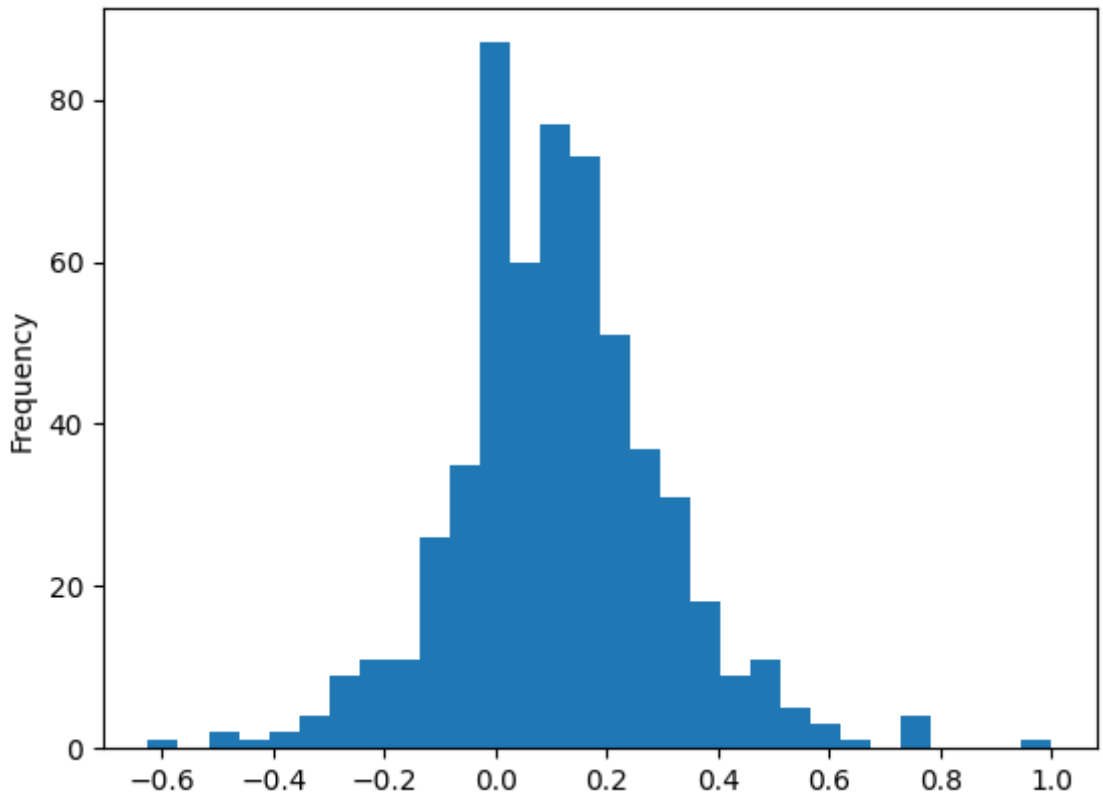
Created a new column that combines the post's title and body for analysis.

In [19]:
```
from textblob import TextBlob

def get_tb_sentiment(text):
    tb = TextBlob(text)
    return tb.sentiment[0]

df['tb_sentiment'] = df['title_and_text'].apply(get_tb_sentiment)
```

Used TextBlob to get the sentiment score for each row, using the title and body text combined.

In [20]:
```
df['tb_sentiment'].plot.hist(bins=30)
```

Out[20]:
```
<AxesSubplot:ylabel='Frequency'>
```



Many rows have a sentiment score of 0, which means it was either *actually* neutral, or (more commonly) did not have any values that were scored in TextBlob. Either way, we can see that the sentiment scores trend toward the positive direction for r/LifeProTips.

In [21]:
```
df['tb_sentiment'].describe()
```

Out[21]:
```
count    570.000000
mean       0.112021
std        0.191109
min       -0.625000
25%        0.000000
50%        0.103255
75%        0.211158
max        1.000000
Name: tb_sentiment, dtype: float64
```

Confirming the above hypothesis, the mean sentiment score for the dataframe is 0.1128. This is a positive score, on a scale of -1 to 1.

Most Common Negative Words

Previewing the lowest sentiment scores:

In [22]:
```
top_scores = df.sort_values('tb_sentiment', ascending=True)[['title_and_t
top_scores
```

Out[22]:

	title_and_text	tb_sentiment
298	LPT: Say weird things on the phone with automa...	-0.625000
345	LPT: If your partner is asthmatic avoid restin...	-0.500000
58	LPT Request: How to stop a bad habit when you ...	-0.500000
215	LPT Request: How to remember to put salt on bo...	-0.462500
185	LPT: if you have access to cold winters you ca...	-0.400000
158	LPT: If you are travelling in Europe and need ...	-0.375000
451	LPT: If you're sick of ads on YouTube, refresh...	-0.347321
221	LPT request: How to get rid of month-old dried...	-0.322222
271	LPT Request: How do I manage my emotions? When...	-0.316667
358	LPT: You don't need to wait for water to boil ...	-0.313333

In [23]:
```
words = ' '.join(df.sort_values("tb_sentiment", ascending=True)["title_an
cleaned_words = [w for w in words if w not in set(stops)]
```

Created a list of the most common words in the lowest scored posts, not including the stopwords.

In [24]:
```
fd = nltk.FreqDist(cleaned_words)
fd.most_common(20)
```

Out[24]:
```
[('get', 6),
 ('water', 5),
 ('cold', 5),
 ('like', 4),
 ('stop', 3),
 ('something', 3),
 ('put', 3),
 ('use', 3),
 ('things', 2),
 ('even', 2),
 ('works', 2),
 ('asthmatic', 2),
 ('arm', 2),
 ('ribs', 2),
 ('lung', 2),
 ('area', 2),
 ('weight', 2),
 ('habit', 2),
 ('time', 2),
 ('salt', 2)]
```

Interestingly, it looks like there are a few posts deemed negative that have "cold" or "water" in the text. Also, I see many body parts, such as "arm", "lung", and "ribs". It seems that the posts that talk about the body tend to be deemed negative.

Comparison and Analysis

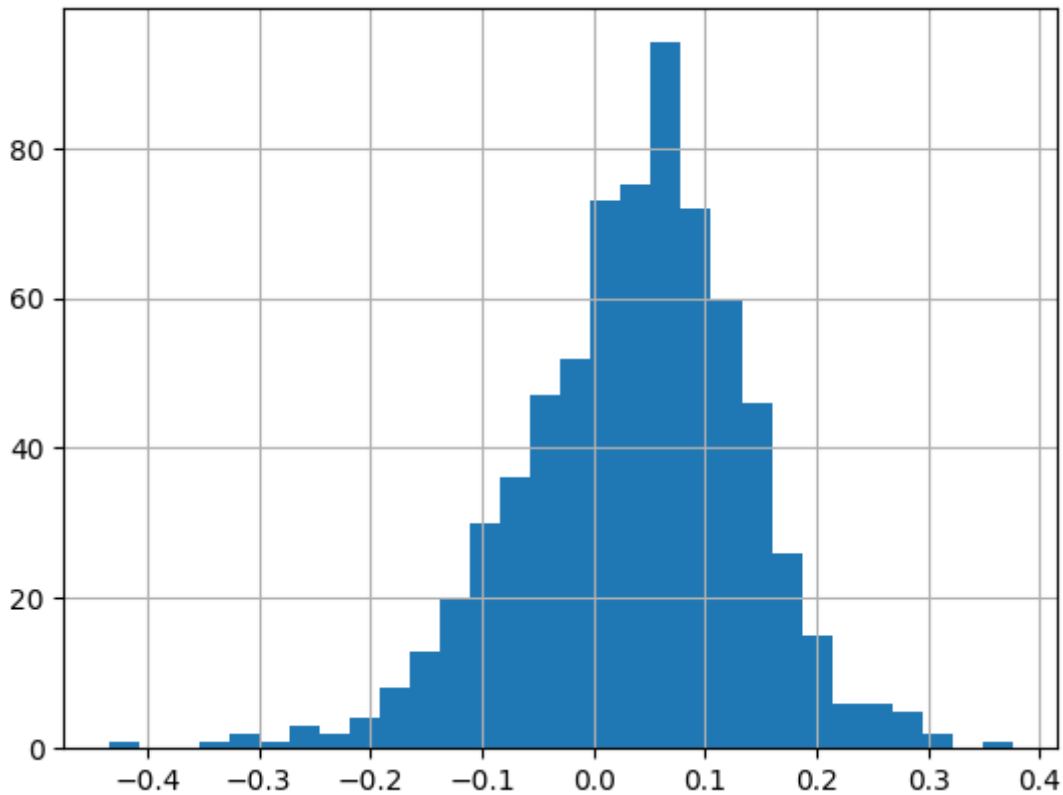
So far, I have just done some exploring on the data. This section will serve as a sort of "conclusion" which will put analysis in context by comparison.

In [25]:
```
aita_subreddit = reddit.subreddit('AmItheAsshole').hot(limit=None)
aita_reddit_data = {
    "title": [],
    "author": [],
    "n_comments": [],
    "score": [],
    "text": [],
    "link": []
}

for post in list(aita_subreddit):
    aita_reddit_data["title"].append(post.title)
    if post.author is None:
        aita_reddit_data["author"].append('')
    else:
        aita_reddit_data["author"].append(post.author.name)

    aita_reddit_data["n_comments"].append(post.num_comments)
    aita_reddit_data["score"].append(post.score)
    aita_reddit_data["text"].append(post.selftext)
    aita_reddit_data["link"].append(post.permalink)

aita_df = pd.DataFrame(aita_reddit_data)
aita_df["title_and_text"] = aita_df["title"] + " " + aita_df["text"]
```

In [26]:
```
def get_tb_sentiment(text):
    tb = TextBlob(text)
    return tb.sentiment[0]

aita_df['tb_sentiment'] = aita_df['title_and_text'].apply(get_tb_sentimen
```

In [27]:
```
aita_df["tb_sentiment"].hist(bins=30)
```

Out[27]:
```
<AxesSubplot:>
```



In [28]:
```
aita_df["tb_sentiment"].describe()
```

Out[28]:
```
count    701.000000
mean       0.035948
std        0.101922
min       -0.434215
25%       -0.025838
50%        0.045922
75%        0.103324
max        0.375455
Name: tb_sentiment, dtype: float64
```

The mean sentiment score for r/LifeProTips was 0.1127, which is a relatively high sentiment score, all things considered. r/AmItheAsshole has a 0.0375 sentiment score, meaning it is positive, but extremely close to neutral. Just based on these numbers, it is evident that r/LPT tends to be a bit more positive than r/AITA, at least in the top posts.

Obviously while analyzing subreddits, it may not be apparently useful, but imagine this tool for analyzing customer sentiment toward products or community engagement in general.